

UNIVERSIDAD DEL VALLE
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación
Análisis de Algoritmos II

Informe de Proyecto

Proyecto de curso

Calculando el plan de riego óptimo de una finca

Estudiantes:	Juan Carlos Cruz Muñoz (1824389), David Alejandro Enciso Gutierrez (2240581), Juan Esteban Rodriguez Valencia (2042282), Estiven Andres Martinez Granados (2179687)
Profesores:	Jesús Alexander Aranda
Monitor:	Samuel Galindo

Cali, Colombia
Mayo de 2026

Índice

1. Introducción	3
2. El Problema del Riego Óptimo	3
2.1. Descripción del Problema	3
2.2. Formalización Matemática	4
3. Solución Mediante Fuerza Bruta	5
3.1. Descripción de la Estrategia	5
3.1.1. Descripción formal de la estrategia	5
3.1.2. Complejidad	5
3.1.3. Corrección	6
3.2. Aplicación del algoritmo	7
3.2.1. Aplicación a los ejemplos del enunciado	7
3.2.2. Análisis de combinaciones y límites computacionales	8
4. Solución Mediante Algoritmo Voraz	8
4.1. Descripción de la Estrategia	8
4.1.1. Correspondencia entre estrategia e implementación	10
4.2. Aplicación del algoritmo y evaluación experimental	12
4.2.1. Aplicación del algoritmo a casos concretos	12
4.2.2. Evaluación experimental	14
4.3. Complejidad y Corrección	20
4.3.1. Síntesis e interpretación de la evidencia	23
5. Solución Mediante Programación Dinámica	25
5.1. Descripción de la Estrategia	25
5.1.1. Paso 1: Caracterizar la estructura de la solución óptima	25
5.1.2. Paso 2: Definir recursivamente el valor de la solución óptima	26
5.1.3. Paso 3: Calcular el valor de la solución óptima	26
5.1.4. Paso 4: Construir una solución óptima	27
5.1.5. Correspondencia entre estrategia e implementación	28
5.2. Análisis de Complejidad en Tiempo y en Espacio	30
5.3. Corrección	32

5.4. Aplicación del algoritmo y evaluación experimental	33
5.4.1. Aplicación a los ejemplos del enunciado	33
5.4.2. Evaluación experimental	34
6. Comparación de las tres soluciones implementadas	34
6.1. Diseño de casos de prueba	35
6.2. Resultados comparativos	36
6.3. Ventajas y desventajas de cada enfoque en la práctica	41
7. Conclusiones	42
Notas de implementación	43

1. Introducción

Este informe desarrolla el problema del riego óptimo planteado en el enunciado del proyecto de curso. El objetivo es construir y analizar tres enfoques para programar el orden de riego de los tablones de una finca: fuerza bruta, algoritmo voraz y programación dinámica. Cada estrategia se estudia desde tres perspectivas: su idea algorítmica, su complejidad y su capacidad para producir una solución óptima.

La implementación fue realizada en C++ y se encuentra disponible en el repositorio del proyecto: <https://github.com/Juan-Carlos-Cruz/Proyecto-I-ADA-II>. La lógica principal reside en `backend/src/algoritmos.cpp`, donde se implementan las funciones `roFB`, `roV` y `roPD`, correspondientes a fuerza bruta, voraz y programación dinámica. El proyecto también incluye una interfaz para cargar archivos de entrada, ejecutar los algoritmos y revisar las salidas obtenidas.

La dificultad principal del problema está en que el costo no solo depende de qué tablón se riega, sino también del instante exacto en que comienza su riego. Esto hace que el orden completo de la programación sea determinante. Como consecuencia, una decisión aparentemente buena en el corto plazo puede afectar negativamente el costo global. Por eso resulta pertinente comparar una estrategia exhaustiva, una heurística voraz y una formulación exacta mediante programación dinámica.

2. El Problema del Riego Óptimo

2.1. Descripción del Problema

Una finca está compuesta por varios tablones y se dispone de un único recurso móvil de riego. Cada tablón tiene cuatro atributos relevantes:

- **Tiempo de supervivencia** (ts_i): número máximo de días que puede pasar sin ser regado antes de sufrir afectación grave.
- **Tiempo de riego** (tr_i): días requeridos para regar completamente el tablón.
- **Prioridad** (p_i): nivel de importancia del tablón, entre 1 y 4.
- **Tiempo de riego perfecto** (rp_i): instante ideal para iniciar el riego del tablón.

Como solo existe un recurso de riego, en cada instante se debe decidir qué tablón atender primero. El problema consiste en hallar un orden de riego que minimice el costo total de afectación de los cultivos. La función de costo del enunciado penaliza tres situaciones:

- si el tablón se riega exactamente en su día perfecto, se obtiene el costo más bajo;
- si se riega antes de superar su límite de supervivencia, pero no en el instante perfecto, el costo es moderado;
- si se riega tarde, la penalización aumenta y además se multiplica por la prioridad del tablón.

En términos prácticos, el problema es una variante de secuenciamiento de tareas con una función objetivo dependiente del tiempo de inicio. Esto significa que el costo de regar un tablón no es fijo, sino que varía según el momento en que se atiende. Por ello, no basta con priorizar los tablonos más urgentes: en algunos casos conviene postergar el riego de un tablón para acercarse a su día perfecto y así reducir su costo, o adelantarlo si esperar incrementa la penalización. En consecuencia, la solución óptima exige evaluar cómo el orden de programación de riego de cada tablón afecta el costo total de la secuencia.

2.2. Formalización Matemática

Sea una finca

$$F = \langle T_0, T_1, \dots, T_{n-1} \rangle,$$

donde cada tablón se define como

$$T_i = \langle ts_i, tr_i, p_i, rp_i \rangle,$$

con $0 \leq rp_i \leq ts_i - tr_i$.

Una programación de riego es una permutación

$$\Pi = \langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle$$

de los índices $\{0, 1, \dots, n-1\}$, que indica el orden en el que se riegan los tablonos.

El instante de inicio del riego de cada tablón depende de los tablonos programados antes que él. Si t_i^Π representa el instante en que empieza a regarse el tablón i bajo la programación Π , entonces:

$$t_{\pi_0}^\Pi = 0, \quad t_{\pi_j}^\Pi = \sum_{k=0}^{j-1} tr_{\pi_k} \quad \text{para } j \geq 1.$$

La contribución al costo del tablón i al empezar a regarse en el instante t_i^Π es:

$$CR_F^\Pi[i] = \begin{cases} ts_i - (t_i^\Pi + tr_i), & \text{si } t_i^\Pi = rp_i, \\ 2(ts_i - (t_i^\Pi + tr_i)), & \text{si } t_i^\Pi \neq rp_i \text{ y } t_i^\Pi \leq ts_i - tr_i, \\ 2p_i((t_i^\Pi + tr_i) - ts_i), & \text{en otro caso.} \end{cases}$$

El costo total de la programación es:

$$CR_F^\Pi = \sum_{i=0}^{n-1} CR_F^\Pi[i].$$

El problema del riego óptimo consiste entonces en encontrar una permutación Π^* tal que:

$$\Pi^* = \arg \min_{\Pi} CR_F^\Pi.$$

3. Solución Mediante Fuerza Bruta

3.1. Descripción de la Estrategia

La estrategia de fuerza bruta resuelve el problema de manera exhaustiva: recorre el conjunto completo de órdenes factibles, evalúa el costo total de cada uno y conserva el mejor encontrado. Como toda solución del problema es una permutación de los n tablonos, esta estrategia examina explícitamente el espacio total de búsqueda.

3.1.1. Descripción formal de la estrategia

Sea

$$F = \langle T_0, T_1, \dots, T_{n-1} \rangle$$

una finca con n tablonos. La función `roFB` parte de la permutación identidad

$$\Pi_0 = \langle 0, 1, \dots, n-1 \rangle$$

y recorre sucesivamente todas sus permutaciones mediante la rutina `next_permutation`. Para cada orden Π , el algoritmo simula el avance del tiempo, calcula el costo CR_F^Π y actualiza la mejor solución si encuentra una permutación de menor costo. Al finalizar el recorrido, vuelve a evaluar la mejor permutación para construir la salida completa con detalles por tablón.

El siguiente pseudocódigo resume esta idea:

```
Funcion roFB(tablonos)
  orden_actual <- [0,1,...,n-1]
  mejor_orden <- orden_actual
  mejor_costo <- +infinito

  repetir
    costo <- evaluar_costo_total(tablonos, orden_actual)
    si costo < mejor_costo
      mejor_costo <- costo
      mejor_orden <- copia(orden_actual)
    mientras next_permutation(orden_actual)

  retornar evaluar_orden_completo(tablonos, mejor_orden)
```

3.1.2. Complejidad

El costo del algoritmo puede justificarse por bloques:

- La inicialización de la permutación identidad cuesta $O(n)$.
- El número de órdenes que se generan es $n!$, pues el espacio de soluciones factibles coincide con el conjunto de todas las permutaciones de los n tablonos.

- Evaluar una permutación cuesta $O(n)$, porque para calcular CR_F^Π basta recorrer una vez los n tableros, actualizando el tiempo acumulado y sumando el costo de cada uno.
- Cada vez que se encuentra una mejor solución, se almacena una copia de la permutación actual, lo cual cuesta $O(n)$. Sin embargo, este costo queda absorbido por el recorrido exhaustivo, cuya fase dominante sigue siendo la evaluación de las $n!$ permutaciones.
- La reevaluación final de `mejor_orden` para construir la salida detallada cuesta $O(n)$ y no modifica la cota asintótica total.

Por tanto, la complejidad en tiempo del algoritmo es:

$$O(n! \cdot n).$$

La complejidad en espacio es $O(n)$, ya que solo se mantienen la permutación actual, la mejor permutación encontrada y un conjunto constante de variables escalares.

Límites computacionales. La dificultad práctica de esta estrategia proviene del crecimiento factorial del número de permutaciones. En particular:

- $10! = 3,628,800$
- $11! = 39,916,800$
- $12! = 479,001,600$
- $13! = 6,227,020,800$

Estos valores muestran que, aun cuando evaluar una permutación individual es lineal, el número de órdenes posibles crece demasiado rápido. En consecuencia, la fuerza bruta es adecuada como referencia exacta para instancias pequeñas, pero se vuelve impráctica para valores moderados de n . En la práctica, el umbral exacto depende del hardware y de la implementación, aunque alrededor de 10 o 12 tableros el costo computacional ya suele ser restrictivo.

3.1.3. Corrección

Proposición. Para toda finca F con n tableros, la función `roFB` devuelve una permutación $\hat{\Pi}$ tal que

$$CR_F^{\hat{\Pi}} = \min_{\Pi} CR_F^{\Pi}.$$

Demostración. Sea

$$\mathcal{P}(F) = \{\Pi_1, \Pi_2, \dots, \Pi_{n!}\}$$

el conjunto de todas las permutaciones factibles de los n tableros. Este conjunto es finito, con cardinalidad $n!$. Como la función de costo CR_F^Π asigna un valor bien definido a cada permutación $\Pi \in \mathcal{P}(F)$, existe al menos una permutación óptima $\Pi^* \in \mathcal{P}(F)$ tal que

$$CR_F^{\Pi^*} \leq CR_F^\Pi \quad \text{para toda } \Pi \in \mathcal{P}(F).$$

La función **roFB** recorre exhaustivamente los elementos de $\mathcal{P}(F)$. En la implementación del proyecto, ese recorrido se realiza mediante **next_permutation**. Durante la enumeración se mantiene el siguiente invariante: después de evaluar las primeras k permutaciones, la variable **mejor_orden** almacena una permutación de costo mínimo entre esas k ya examinadas.

La prueba del invariante es inmediata por inducción sobre k :

- Para $k = 1$, tras evaluar la primera permutación, **mejor_orden** queda igual a esa única solución examinada, y por tanto es óptima dentro del prefijo considerado.
- Supóngase cierto para k . Al evaluar la permutación $k + 1$, el algoritmo compara su costo con el de **mejor_orden**. Si la nueva permutación es mejor, la reemplaza; si no, conserva la anterior. En ambos casos, al finalizar la iteración, **mejor_orden** sigue representando una permutación de costo mínimo entre las primeras $k + 1$.

Al terminar el recorrido de las $n!$ permutaciones, el invariante implica que **mejor_orden** tiene costo mínimo en todo $\mathcal{P}(F)$. En consecuencia,

$$CR_F^{\text{mejor_orden}} = \min_{\Pi \in \mathcal{P}(F)} CR_F^\Pi.$$

La evaluación final de **mejor_orden** solo reconstruye la salida detallada y no altera la permutación elegida ni su costo. Por tanto, **roFB** siempre devuelve una solución óptima.

3.2. Aplicación del algoritmo

3.2.1. Aplicación a los ejemplos del enunciado

Tomemos las dos fincas pequeñas del enunciado:

$$F_1 = \langle \langle 10, 3, 4, 0 \rangle, \langle 6, 3, 3, 1 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 10, 4, 2, 5 \rangle \rangle$$

$$F_2 = \langle \langle 9, 3, 4, 0 \rangle, \langle 5, 3, 3, 2 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 6, 4, 2, 1 \rangle \rangle.$$

Como ambas instancias tienen $n = 5$, la fuerza bruta evalúa exactamente

$$5! = 120$$

permutaciones en cada caso. Al recorrer exhaustivamente ese espacio, la función **roFB** encuentra las siguientes soluciones óptimas:

Finca	Permutaciones evaluadas	Permutación óptima	Costo óptimo
F_1	120	$\langle 2, 1, 0, 3, 4 \rangle$	20
F_2	120	$\langle 2, 1, 0, 3, 4 \rangle$	32

Tabla 1: Resultado de la búsqueda exhaustiva sobre los ejemplos pequeños del enunciado.

En F_1 , la mejor permutación encontrada tiene costo 20; en F_2 , el mejor costo es 32. En ambos casos, la salida coincide con la referencia exacta que luego se usa para evaluar al voraz y para contrastar la programación dinámica. Esto ilustra el papel práctico de la fuerza bruta en el proyecto: para instancias pequeñas, sirve como mecanismo de validación exacta; para instancias grandes, su crecimiento factorial la vuelve inviable.

3.2.2. Análisis de combinaciones y límites computacionales

Para profundizar en la aplicación del algoritmo y evaluar el comportamiento de la fuerza bruta en escenarios más complejos planteados por el equipo ($n = 10$ y $n = 12$), se analiza cuantitativamente la cantidad de combinaciones (permutaciones) evaluadas en función del número de tablonos. Debido a que el espacio de búsqueda crece de forma factorial ($n!$), un incremento lineal en el tamaño de la finca desencadena una explosión combinatoria.

La siguiente tabla resume el total de permutaciones periódicas que la función `roFB` debe examinar exhaustivamente para determinar el mínimo global:

Tamaño de la finca (n)	Permutaciones totales evaluadas ($n!$)
5	120
8	40,320
10	3,628,800
12	479,001,600

Tabla 2: Escalabilidad del espacio de búsqueda y total de combinaciones evaluadas.

Esta aplicación numérica evidencia por qué el enfoque exhaustivo responde de inmediato con las instancias base (120 combinaciones), pero se vuelve impracticable al alcanzar los 10 o 12 tablonos. Procesar casi 480 millones de combinaciones para $n = 12$ satura rápidamente el tiempo de CPU en entornos convencionales, lo que justifica la implementación de estrategias más eficientes como el enfoque voraz o la programación dinámica.

4. Solución Mediante Algoritmo Voraz

4.1. Descripción de la Estrategia

La implementación voraz del proyecto corresponde a la función `roV`. El algoritmo recibe una finca

$$F = \langle T_0, \dots, T_{n-1} \rangle$$

y construye una permutación Π_V que define el orden de riego. Para cada tablón i calcula el valor

$$\rho_i = \frac{tr_i}{p_i},$$

y luego ordena los tablonos de menor a mayor según ρ_i . Si dos tablonos empatan en ese valor, se desempata usando el margen

$$ts_i - tr_i,$$

privilegiando primero el tablón con menor margen.

Intuición. Si dos tablonos ya están en el caso tardío, el costo de regar i en el instante t queda dominado por

$$2p_i((t + tr_i) - ts_i).$$

Aquí aparecen dos fuerzas: un tr_i grande retrasa más a los demás, y un p_i grande hace más caro ese retraso. Por eso el cociente $\frac{tr_i}{p_i}$ intenta equilibrar duración y penalización.

Comparación local. Para justificar formalmente el criterio voraz, comparemos dos tablonos i y j , suponiendo que ambos ya están en caso 3 en el instante t . En ese contexto se evalúan dos únicos órdenes posibles: regar primero i y luego j , o regar primero j y luego i .

Si se riega primero i , el costo conjunto es:

$$C(i \rightarrow j) = 2p_i(t + tr_i - ts_i) + 2p_j(t + tr_i + tr_j - ts_j).$$

Si se riega primero j , el costo conjunto es:

$$C(j \rightarrow i) = 2p_j(t + tr_j - ts_j) + 2p_i(t + tr_j + tr_i - ts_i).$$

Al restar ambos costos se obtiene:

$$\Delta_{ij} = C(i \rightarrow j) - C(j \rightarrow i) = 2(p_j tr_i - p_i tr_j).$$

Para que sea mejor poner i antes que j , esta diferencia debe ser menor o igual que cero:

$$\Delta_{ij} \leq 0.$$

Sustituyendo la expresión anterior:

$$2(p_j tr_i - p_i tr_j) \leq 0.$$

De ahí se obtiene:

$$p_j tr_i \leq p_i tr_j.$$

Y como las prioridades son positivas, se puede dividir por $p_i p_j$ y concluir que

$$\frac{tr_i}{p_i} \leq \frac{tr_j}{p_j}.$$

Intuición del desempate. Cuando dos tablonos comparten el mismo cociente $\frac{tr}{p}$, se desempata por la holgura $ts_i - tr_i$: el umbral de tiempo de inicio de riego a partir del cual el tablón entra al caso 3. El tablón con menor holgura tiene ese umbral más bajo, por lo que es más fácil que cualquier espera lo lleve al caso tardío e incrementa su costo. Por eso se atiende primero. Este criterio no se demuestra óptimo formalmente, pero es coherente con la función de costo del problema.

Desde el punto de vista computacional, la estrategia tiene tres pasos:

Paso	Operación	Complejidad
1	Calcular $\frac{tr_i}{p_i}$ y $ts_i - tr_i$ para cada tablón	$O(n)$
2	Ordenar por $\frac{tr}{p}$ y desempatar por $ts - tr$	$O(n \log n)$
3	Simular la permutación y calcular los costos	$O(n)$

Tabla 3: Desglose operacional del algoritmo voraz.

4.1.1. Correspondencia entre estrategia e implementación

El siguiente fragmento no se incluye para deducir la complejidad línea por línea, sino para mostrar que la implementación en C++ preserva exactamente el criterio voraz descrito antes. La implementación de `roV` no reordena directamente los tablonos, sino sus índices. Para simplificar la escritura, definamos

$$\rho_i = \frac{tr_i}{p_i}, \quad h_i = ts_i - tr_i.$$

Entonces, para dos tablonos i y j , el comparador usado por `sort` aplica estas reglas en orden:

1. Si $\rho_i < \rho_j$, entonces i va antes que j .
2. Si $\rho_i = \rho_j$, va primero el tablón con menor holgura, es decir, el que tenga menor h .
3. Si también empatan en holgura, va primero el de menor índice original.

Como el código evita divisiones en punto flotante, la comparación principal no se implementa calculando cocientes decimales, sino mediante producto cruzado:

$$\frac{tr_i}{p_i} < \frac{tr_j}{p_j} \iff tr_i p_j < tr_j p_i.$$

El siguiente fragmento muestra el núcleo de esa implementación:

```
vector<int> orden(cantidad_tablonos);
iota(orden.begin(), orden.end(), 0);

sort(orden.begin(), orden.end(), [&](int indice_a, int indice_b) {
    const Tablon& a = tablonos[indice_a];
    const Tablon& b = tablonos[indice_b];

    long long ratio_a = 1LL * a.tiempo_riego * b.prioridad;
    long long ratio_b = 1LL * b.tiempo_riego * a.prioridad;
    if (ratio_a != ratio_b) return ratio_a < ratio_b;

    const int margen_a = a.tiempo_supervivencia - a.tiempo_riego;
    const int margen_b = b.tiempo_supervivencia - b.tiempo_riego;
    if (margen_a != margen_b) return margen_a < margen_b;

    return indice_a < indice_b;
});
```

Brevemente, cada bloque del fragmento cumple una función específica dentro de la estrategia voraz:

- `vector<int>orden(cantidad_tablones);` reserva el arreglo donde se guardará la permutación de índices que luego se ordenará.
- `iota(orden.begin(), orden.end(), 0);` inicializa ese arreglo como $\langle 0, 1, \dots, n-1 \rangle$, es decir, con el orden original de los tablones.
- `sort(...)` aplica el comparador voraz sobre esos índices para construir la permutación final.
- `const Tablon& a = tablones[indice_a];` recupera el primer tablón involucrado en la comparación.
- `const Tablon& b = tablones[indice_b];` recupera el segundo tablón involucrado en la comparación.
- `ratio_a` y `ratio_b` implementan el producto cruzado $tr_i p_j$ y $tr_j p_i$, evitando dividir en punto flotante.
- `if (ratio_a != ratio_b) return ratio_a < ratio_b;` decide el orden principal según la regla $\frac{tr_i}{p_i} < \frac{tr_j}{p_j}$.
- `margen_a` y `margen_b` calculan la holgura $ts - tr$ de cada tablón para usarla como segundo criterio.
- `if (margen_a != margen_b) return margen_a < margen_b;` da prioridad al tablón con menor margen cuando la razón principal empata.
- `return indice_a < indice_b;` resuelve el empate final conservando el orden por índice y hace determinista la salida del algoritmo.

Ejemplo. Si T_i tiene $(tr_i, p_i) = (2, 4)$ y T_j tiene $(tr_j, p_j) = (3, 2)$, el código no compara $2/4$ con $3/2$, sino

$$2 \cdot 2 = 4 \quad \text{y} \quad 3 \cdot 4 = 12.$$

Como $4 < 12$, decide $i \prec j$. Si dos tablones empatan en $\frac{tr}{p}$, por ejemplo $(ts_i, tr_i, p_i) = (6, 2, 2)$ y $(ts_j, tr_j, p_j) = (9, 3, 3)$, entonces ambos tienen razón 1 y el desempate pasa a la holgura:

$$ts_i - tr_i = 4 \quad < \quad ts_j - tr_j = 6.$$

Por eso el algoritmo coloca primero a i .

En conjunto, el fragmento confirma que la implementación concreta preserva exactamente la estructura del algoritmo descrito antes: construir una permutación de índices, ordenarla según el criterio $\frac{tr}{p}$, desempatar por holgura y resolver los empates finales de forma determinista por índice original.

4.2. Aplicación del algoritmo y evaluación experimental

4.2.1. Aplicación del algoritmo a casos concretos

Aplicación a los ejemplos del enunciado. Para las dos fincas de la Sección 2.3 del enunciado, el criterio voraz produce la misma permutación:

$$\Pi_V = \langle 0, 1, 3, 2, 4 \rangle.$$

Ejemplo 1. Para

$$F_1 = \langle \langle 10, 3, 4, 0 \rangle, \langle 6, 3, 3, 1 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 10, 4, 2, 5 \rangle \rangle$$

los valores relevantes son:

Tablón	ts	tr	p	rp	$\frac{tr}{p}$	$ts - tr$
T_0	10	3	4	0	0.75	7
T_1	6	3	3	1	1.00	3
T_2	2	2	1	0	2.00	0
T_3	8	1	1	6	1.00	7
T_4	10	4	2	5	2.00	6

Tabla 4: Valores usados por el voraz en la finca F_1 .

El orden voraz es

$$T_0 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4.$$

Al simular esa programación se obtiene:

Paso	Tablón	t_{inicio}	Caso aplicado	Costo
1	T_0	0	caso 1: $10 - (0 + 3)$	7
2	T_1	3	caso 2: $2(6 - (3 + 3))$	0
3	T_3	6	caso 1: $8 - (6 + 1)$	1
4	T_2	7	caso 3: $2 \cdot 1 \cdot ((7 + 2) - 2)$	14
5	T_4	9	caso 3: $2 \cdot 2 \cdot ((9 + 4) - 10)$	12

Tabla 5: Evaluación detallada del voraz sobre F_1 .

$$CR_{F_1}^{\Pi_V} = 7 + 0 + 1 + 14 + 12 = 34.$$

La programación óptima por fuerza bruta es

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle$$

con costo 20. La diferencia se explica porque el voraz posterga T_2 , que tiene holgura cero ($ts_2 - tr_2 = 0$), y lo condena a caer en caso 3.

Por tanto, la solución voraz para F_1 **no es óptima**.

Paso	Tablón	t_{inicio}	Caso aplicado	Costo
1	T_0	0	caso 1: $9 - (0 + 3)$	6
2	T_1	3	caso 3: $2 \cdot 3 \cdot ((3 + 3) - 5)$	6
3	T_3	6	caso 1: $8 - (6 + 1)$	1
4	T_2	7	caso 3: $2 \cdot 1 \cdot ((7 + 2) - 2)$	14
5	T_4	9	caso 3: $2 \cdot 2 \cdot ((9 + 4) - 6)$	28

Tabla 6: Evaluación detallada del voraz sobre F_2 .

Ejemplo 2. Para

$$F_2 = \langle \langle 9, 3, 4, 0 \rangle, \langle 5, 3, 3, 2 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 6, 4, 2, 1 \rangle \rangle$$

se obtiene la misma permutación voraz. La evaluación queda:

$$CR_{F_2}^{\Pi_V} = 6 + 6 + 1 + 14 + 28 = 55.$$

El óptimo es nuevamente

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle$$

con costo 32. El patrón de falla es el mismo: un tablón con margen nulo queda demasiado atrás por tener una prioridad baja y, por tanto, un cociente $\frac{tr}{p}$ alto.

Por tanto, la solución voraz para F_2 **no es óptima**.

Para complementar los dos ejemplos del enunciado sin reutilizar la batería de pruebas del repositorio, se diseñaron cuatro fincas adicionales pequeñas:

Ejemplo 3. Para

$$F_3 = \langle \langle 11, 3, 3, 3 \rangle, \langle 13, 3, 3, 2 \rangle, \langle 6, 3, 1, 0 \rangle, \langle 5, 2, 2, 2 \rangle, \langle 5, 4, 4, 1 \rangle \rangle$$

el voraz produce

$$\Pi_V = \langle 4, 3, 0, 1, 2 \rangle \quad \text{con} \quad CR_{F_3}^{\Pi_V} = 30.$$

La salida óptima de referencia es

$$\Pi^* = \langle 4, 3, 0, 1, 2 \rangle \quad \text{con costo} \quad 30.$$

Por tanto, la solución voraz para F_3 **sí es óptima**.

Ejemplo 4. Para

$$F_4 = \langle \langle 5, 4, 4, 0 \rangle, \langle 13, 4, 2, 6 \rangle, \langle 8, 7, 2, 0 \rangle, \langle 9, 1, 1, 1 \rangle, \langle 8, 2, 2, 5 \rangle \rangle$$

el voraz produce

$$\Pi_V = \langle 0, 4, 3, 1, 2 \rangle \quad \text{con} \quad CR_{F_4}^{\Pi_V} = 53.$$

La salida óptima de referencia es

$$\Pi^* = \langle 0, 4, 1, 3, 2 \rangle \quad \text{con costo} \quad 52.$$

Por tanto, la solución voraz para F_4 **no es óptima**.

Ejemplo 5. Para

$$F_5 = \langle \langle 11, 4, 3, 0 \rangle, \langle 8, 5, 4, 2 \rangle, \langle 6, 2, 3, 0 \rangle, \langle 15, 9, 2, 1 \rangle, \langle 5, 1, 4, 0 \rangle \rangle$$

el voraz produce

$$\Pi_V = \langle 4, 2, 1, 0, 3 \rangle \quad \text{con} \quad CR_{F_5}^{\Pi_V} = 40.$$

La salida óptima de referencia es

$$\Pi^* = \langle 2, 4, 1, 0, 3 \rangle \quad \text{con costo} \quad 38.$$

Por tanto, la solución voraz para F_5 **no es óptima**.

Ejemplo 6. Para

$$F_6 = \langle \langle 12, 6, 4, 5 \rangle, \langle 7, 2, 2, 5 \rangle, \langle 9, 4, 3, 2 \rangle, \langle 8, 8, 4, 0 \rangle, \langle 13, 6, 4, 3 \rangle \rangle$$

el voraz produce

$$\Pi_V = \langle 1, 2, 0, 4, 3 \rangle \quad \text{con} \quad CR_{F_6}^{\Pi_V} = 197.$$

La salida óptima de referencia es

$$\Pi^* = \langle 2, 1, 0, 4, 3 \rangle \quad \text{con costo} \quad 196.$$

Por tanto, la solución voraz para F_6 **no es óptima**.

Caso adicional	n	Costo voraz	Costo óptimo	Diferencia	¿Es óptima?
Ejemplo 3	5	30	30	0	Sí
Ejemplo 4	5	53	52	1	No
Ejemplo 5	5	40	38	2	No
Ejemplo 6	5	197	196	1	No

Tabla 7: Resumen de cuatro entradas adicionales diseñadas para el análisis del algoritmo voraz.

4.2.2. Evaluación experimental

Experimento 1: batería de pruebas del repositorio. Para complementar el análisis, aquí se reporta el conjunto completo de los 18 casos de la batería de pruebas del repositorio que sí cuentan con entrada y salida de referencia. Mostrar todos los casos permite respaldar directamente las estadísticas agregadas y ver cómo varía el desempeño del voraz según el tamaño y la estructura de cada finca.

Test	n	Costo óptimo	Costo voraz	Diferencia	% exceso	¿Voraz = óptimo?
1	5	44	59	15	34.09	No
2	5	126	128	2	1.59	No
3	5	38	57	19	50.00	No
4	5	26	52	26	100.00	No
5	6	193	193	0	0.00	Sí
6	6	256	288	32	12.50	No
7	7	67	80	13	19.40	No
8	7	294	302	8	2.72	No
9	8	143	178	35	24.48	No
10	8	522	534	12	2.30	No
11	10	698	762	64	9.17	No
12	10	526	638	112	21.29	No
13	15	1276	1297	21	1.65	No
14	15	2460	2468	8	0.33	No
15	15	1030	1080	50	4.85	No
16	20	3758	3816	58	1.54	No
17	20	2861	2894	33	1.15	No
18	20	1512	1590	78	5.16	No

Tabla 8: Comparación entre costo óptimo y costo voraz en los 18 casos del repositorio con salida de referencia.

Elemento	Valor / fórmula	Lectura rápida
Casos evaluados	18	Tests con entrada y salida de referencia.
Coincidencias exactas	$1/18 = 5,56\%$	Veces en que el voraz iguala al óptimo.
Diferencia promedio	32,56	Promedio de d_k .
Exceso relativo promedio	16,23%	Promedio de e_k .
Error absoluto por caso	$d_k = C_V^{(k)} - C^{*(k)} $	Distancia entre el costo voraz y el costo óptimo del caso k .
Error relativo por caso	$e_k = 100 \cdot \frac{d_k}{C^{*(k)}}$	Ese mismo error, pero expresado como porcentaje del óptimo.

Tabla 5.1. Resumen estadístico y fórmulas usadas para interpretar la Tabla 5.

Aquí $C_V^{(k)}$ es el costo obtenido por el voraz en el caso k y $C^{*(k)}$ es el costo óptimo de ese mismo caso. En esta batería, el peor caso absoluto fue el **test12** (638 frente a 526) y el peor caso relativo fue el **test4**, donde el voraz duplicó el costo óptimo (52 frente a 26).

Experimento 2: bloque mixto aleatorio de 5000 fincas con $n = 20$. La batería de pruebas permite exhibir contraejemplos y observar con detalle algunos casos concretos, pero no basta para estimar cómo se comporta el voraz de manera típica. Por eso se realiza una validación empírica adicional sobre una muestra grande: la idea es comprobar si los patrones observados antes se mantienen cuando se consideran muchas fincas distintas, obtener promedios más estables y aislar el análisis en un tamaño ya exigente ($n = 20$).

Además de la batería de pruebas, se analizó el bloque **n20** definido por la versión actual del script `generar_tests_aleatorios.py`. En su estado actual, el generador produce únicamente instancias con $n = 20$, con un total de 5000 fincas y semilla fija 20260513. Cada tablón se construye a partir de un perfil aleatorio:

Cómo se genera cada perfil. En todos los perfiles se sigue la misma plantilla:

$$\text{margen} = ts - tr, \quad tr = ts - \text{margen}, \quad rp \in [0, \text{margen}].$$

En el script, esta variable aparece con el nombre **slack**. La diferencia entre perfiles está en cómo se sortean ts , margen y p :

- **Crítico:** se sortea $ts \in [5, 15]$, se fija $\text{margen} = 0$ y se sortea $p \in [1, 4]$. Luego $tr = ts$ y necesariamente $rp = 0$. Intención: modelar tableros que no admiten espera.
- **Urgente:** se sortea $ts \in [5, 10]$, luego $\text{margen} \in [1, \min(3, ts - 1)]$ y $p \in [3, 4]$. Después se calcula $tr = ts - \text{margen}$ y se elige $rp \in [0, \text{margen}]$. Intención: hacer que pocos días de retraso ya cuesten caro.
- **Balanceado:** se sortea $ts \in [7, 13]$, luego $\text{margen} \in [2, \min(6, ts - 1)]$ y $p \in [1, 4]$. Después se calcula $tr = ts - \text{margen}$ y se elige $rp \in [0, \text{margen}]$. Intención: representar un punto medio donde compiten urgencia y día perfecto.
- **Relajado:** se sortea $ts \in [10, 15]$, luego $\text{margen} \in [7, \min(13, ts - 1)]$ y $p \in [1, 4]$. Después se calcula $tr = ts - \text{margen}$ y se elige $rp \in [0, \text{margen}]$. Intención: dar mucho margen para que el día perfecto pese más que la urgencia inmediata.
- **Largo plazo:** se sortea $ts \in [20, 100]$, luego $\text{margen} \in [10, \min(50, ts - 1)]$ y $p \in [1, 4]$. Después se calcula $tr = ts - \text{margen}$ y se elige $rp \in [0, \text{margen}]$. Intención: mezclar horizontes temporales largos con otros más tensos.

Relación con la función de costo. Si t es el instante en que empieza el riego, entonces:

$$t = rp \Rightarrow \text{caso 1}, \quad t \leq \text{margen} \text{ y } t \neq rp \Rightarrow \text{caso 2}, \quad t > \text{margen} \Rightarrow \text{caso 3}.$$

Así, un margen pequeño produce tableros críticos o urgentes; un margen grande deja más espacio para que el día perfecto rp influya. La prioridad p solo agrava el caso 3.

Ejemplos por perfil.

- **Crítico:** $ts = 8$, $\text{margen} = 0$, luego $tr = 8$ y $rp = 0$. Si se inicia en $t = 0$, cae en el caso 1; cualquier retraso lo lleva de inmediato al caso 3.
- **Urgente:** $ts = 9$, $\text{margen} = 2$, luego $tr = 7$ y se puede tomar $rp = 1$. Si $t = 1$, caso 1; si $t = 2$, caso 2; si $t > 2$, caso 3.
- **Balanceado:** $ts = 11$, $\text{margen} = 4$, luego $tr = 7$ y se puede tomar $rp = 2$. Si $t = 2$, caso 1; si $t = 4$, caso 2; si $t > 4$, caso 3.
- **Relajado:** $ts = 14$, $\text{margen} = 9$, luego $tr = 5$ y se puede tomar $rp = 6$. Si $t = 6$, caso 1; si $t = 8$, caso 2; si $t > 9$, caso 3.
- **Largo plazo:** $ts = 40$, $\text{margen} = 20$, luego $tr = 20$ y se puede tomar $rp = 12$. Si $t = 12$, caso 1; si $t = 18$, caso 2; si $t > 20$, caso 3.

Distribución nominal del generador. Cada tablón se genera de forma independiente con las siguientes probabilidades:

- 10 % crítico;
- 30 % urgente;
- 25 % balanceado;
- 20 % relajado;
- 15 % largo plazo.

Estos porcentajes no vienen impuestos por el enunciado; se eligieron para construir una muestra variada pero no extrema. La idea es que los perfiles tensos dominen ligeramente la mezcla, porque son los que más rápido exponen fallas del voraz, sin eliminar por completo los perfiles con mucha espera, donde el día perfecto *rp* pesa más.

- el 10 % crítico asegura casos borde, pero evita que toda la muestra quede dominada por instancias demasiado rígidas;
- el 30 % urgente y el 25 % balanceado concentran la mayor parte de la muestra en la zona más informativa, donde compiten urgencia, prioridad y día perfecto;
- el 20 % relajado introduce suficientes tablonos posponibles para que el experimento no favorezca artificialmente al voraz;
- el 15 % largo plazo agrega heterogeneidad temporal sin volver la muestra irrealmente dispersa.

Por qué el generador es válido. La generación siempre preserva las restricciones del enunciado:

$$ts \geq tr, \quad 1 \leq p \leq 4, \quad 0 \leq rp \leq ts - tr.$$

Además, por construcción:

- se cubren simultáneamente casos borde ($ts = tr$), tablonos urgentes, balanceados y relajados;
- no se copian literalmente los casos de la batería de pruebas del proyecto;
- se obtiene una familia sintética coherente con el enunciado y con el estilo de datos observado en la batería real.

Por qué se fijó $n = 20$. Esta decisión tiene dos ventajas metodológicas:

- aísla el comportamiento del criterio voraz en un tamaño ya exigente, sin mezclarlo con el efecto del crecimiento de n ;
- todavía permite resolver todas las instancias por programación dinámica, de modo que cada salida de `roV` se compara contra un óptimo exacto.

Tamaño y validez de la muestra. El bloque usa 5000 fincas de $n = 20$, es decir, 100000 tablones. Ese tamaño basta para que el experimento no dependa de unos pocos casos atípicos y, al mismo tiempo, permite comparar cada salida de roV contra un óptimo exacto calculado por programación dinámica.

La mezcla generada también es consistente con el diseño del experimento. En promedio, cada finca contiene 2 tablones críticos, 6 urgentes, 5 balanceados, 4 relajados y 3 de largo plazo. Las frecuencias observadas quedaron prácticamente iguales a las nominales: 9,994 % crítico, 29,858 % urgente, 24,838 % balanceado, 20,239 % relajado y 15,071 % largo plazo, con una diferencia máxima de solo 0,24 puntos porcentuales entre lo esperado y lo observado.

Además, la muestra no está sesgada hacia instancias “fáciles”: el 99,92 % de las fincas contiene al menos un urgente, el 99,54 % al menos un balanceado, el 95,92 % al menos un tablón de largo plazo, y en el 100 % aparece al menos un tablón con margen ≤ 3 y prioridad 3 o 4. Por eso este bloque sirve para evaluar al voraz en un entorno repetible y exigente.

Conclusión metodológica. En síntesis, el experimento es empíricamente sólido porque el generador respeta las restricciones formales, cubre perfiles distintos en proporciones controladas e introduce trampas heurísticas relevantes; además, usa semilla fija y compara siempre contra el costo óptimo real. Bajo esas condiciones, los resultados no dependen de coincidencias fortuitas.

Los resultados del criterio voraz sobre el bloque mixto de $n = 20$, construido con la mezcla de perfiles definida arriba, se resumen en la tabla siguiente:

Métrica	Resultado en 5000 casos con $n = 20$
Coincidencias exactas con el óptimo	219/5000 = 4,38 %
Diferencia absoluta promedio	66.94
Exceso relativo promedio frente al óptimo	1,97 %
Peor diferencia absoluta observada	382

Tabla 9: Resultados del criterio voraz con $n = 20$ sobre el bloque mixto generado a partir de los perfiles definidos.

Resumen estadístico descriptivo. En las 5000 fincas analizadas, el voraz alcanzó exactamente el mismo costo óptimo en 219 casos (4,38 %). Cuando no alcanzó ese costo, la diferencia absoluta promedio fue de 66.94 unidades y el exceso relativo promedio frente al óptimo fue de apenas 1,97 %. El peor desvío absoluto observado en todo el bloque fue 382.

Interpretación del bloque mixto. En conjunto, esto indica que el voraz rara vez alcanza exactamente el costo óptimo cuando conviven perfiles distintos, aunque el costo adicional que pierde suele mantenerse bajo. Su debilidad principal no está tanto en el valor promedio final, sino en la dificultad para ordenar correctamente tablones con urgencias, prioridades y días perfectos heterogéneos.

Experimento 3: aislamiento por perfiles. El bloque mixto anterior sirve para medir el comportamiento global del criterio, pero no permite ver qué perfil explica cada error. Por eso, a continuación se realiza un **experimento de aislamiento** por perfil. El manejo del análisis cambia así: en lugar de mezclar tipos de tablonos, se generan cinco bloques separados de 5000 fincas con $n = 20$, cada uno compuesto al 100 % por un solo perfil (crítico, urgente, balanceado, largo plazo o relajado), manteniendo la misma semilla fija 20260513 y comparando siempre contra el óptimo exacto. En cada bloque se reportan las mismas tres medidas: diferencia promedio, exceso relativo promedio y tasa de coincidencia exacta en costo con el óptimo. La tabla siguiente muestra cómo cambia el rendimiento del voraz en esos escenarios puros:

Perfil exclusivo en la Finca ($n = 20$)	Diferencia promedio	Exceso relativo %	Tasa de coincidencia exacta en costo
Fincas 100 % Críticos	0.00	0.00 %	5000/5000 (100 %)
Fincas 100 % Urgentes	2.83	0.05 %	1735/5000 (≈ 35 %)
Fincas 100 % Balanceados	11.13	0.35 %	533/5000 (≈ 11 %)
Fincas 100 % Largo plazo	180.25	1.41 %	25/5000 (≈ 0.5 %)
Fincas 100 % Relajados	62.37	5.53 %	4/5000 (≈ 0.1 %)

Tabla 10: Resultados del experimento de aislamiento por perfil sobre bloques puros de 5000 fincas con $n = 20$.

Resumen estadístico descriptivo. El experimento de aislamiento muestra un contraste nítido entre perfiles. En fincas 100 % críticas, el voraz alcanzó el costo óptimo en los 5000 casos. En fincas 100 % urgentes, todavía mantuvo un desempeño muy alto: diferencia promedio 2.83, exceso relativo 0,05 % y 1735 coincidencias exactas en costo. En cambio, al pasar a perfiles con más espera, esa tasa de costo óptimo cayó a 533/5000 en balanceado, 25/5000 en largo plazo y 4/5000 en relajado.

Interpretación del aislamiento por perfiles. La conclusión es que el voraz funciona muy bien cuando el perfil obliga a decidir rápido, pero pierde confiabilidad cuando crece la holgura. Aun así, esa pérdida no siempre implica un gran daño económico: en balanceado y largo plazo la tasa de costo óptimo cae mucho, pero el exceso relativo sigue siendo moderado. El perfil verdaderamente más adverso es relajado, porque allí el día perfecto pesa más que la urgencia inmediata. En otras palabras, el aislamiento confirma que la principal debilidad del criterio aparece cuando debe ordenar tablonos muy posponibles.

Experimento 4: comparación de criterios voraces. Para justificar inequívocamente la selección de $\frac{tr}{p}$ frente a intuiciones alternativas que a priori podrían parecer razonables (como atender lo más urgente, de mayor prioridad o el día perfecto), se ejecutó un laboratorio comparando todas las 20 combinaciones ordenadas implementadas en `backend/src/voraz_criterios.cpp`: 5 criterios posibles como clave principal y 4 como desempate. El ranking sobre el nuevo conjunto mixto de 5000 instancias fue el siguiente:

Tabla 11: Comparación completa de las 20 combinaciones de criterios voraces sobre el bloque mixto oficial de 5000 fincas con $n = 20$.

Principal	Desempate	% dif. promedio	Dif. promedio	Exactas
$\frac{tr}{p}$	$ts - tr$	1.97 %	66.94	219/5000
$\frac{tr}{p}$	supervivencia	1.98 %	67.12	215/5000
$\frac{tr}{p}$	día perfecto	2.08 %	70.56	168/5000
$\frac{tr}{p}$	prioridad alta	2.20 %	74.74	143/5000
supervivencia	$\frac{tr}{p}$	26.14 %	924.86	0/5000
supervivencia	prioridad alta	27.65 %	978.98	0/5000
supervivencia	día perfecto	31.47 %	1115.38	0/5000
supervivencia	$ts - tr$	32.48 %	1150.57	0/5000
prioridad alta	$\frac{tr}{p}$	43.93 %	1712.93	0/5000
prioridad alta	supervivencia	48.67 %	1882.31	0/5000
prioridad alta	$ts - tr$	60.13 %	2315.99	0/5000
prioridad alta	día perfecto	63.64 %	2455.57	0/5000
$ts - tr$	$\frac{tr}{p}$	69.78 %	2517.59	0/5000
$ts - tr$	supervivencia	70.82 %	2556.30	0/5000
$ts - tr$	prioridad alta	71.65 %	2587.62	0/5000
$ts - tr$	día perfecto	74.73 %	2699.56	0/5000
día perfecto	$\frac{tr}{p}$	77.50 %	2810.43	0/5000
día perfecto	supervivencia	80.84 %	2931.55	0/5000
día perfecto	prioridad alta	83.28 %	3034.64	0/5000
día perfecto	$ts - tr$	90.03 %	3270.25	0/5000

Resumen estadístico descriptivo. El ranking muestra una separación muy marcada entre familias de criterios. Las cuatro variantes con $\frac{tr}{p}$ como criterio principal ocupan los cuatro primeros lugares, con exceso relativo promedio entre 1.97 % y 2.20 %, diferencia promedio entre 66.94 y 74.74, y entre 143 y 219 coincidencias exactas en costo con el óptimo. La mejor combinación fue $\frac{tr}{p}$ con desempate por $ts - tr$, seguida muy de cerca por $\frac{tr}{p}$ con supervivencia. En cambio, la mejor variante que ya no usa $\frac{tr}{p}$ como criterio principal salta a 26.14 % de exceso promedio y 0 coincidencias exactas en costo, mientras que el peor caso del barrido fue día perfecto con desempate por $ts - tr$, con 90.03 %.

Interpretación de la comparación de criterios. La conclusión es que el desempate sí influye, pero su efecto es secundario frente a la elección del criterio principal. Lo determinante es conservar $\frac{tr}{p}$ como clave base, porque apenas se reemplaza por urgencia, prioridad, supervivencia o día perfecto, el rendimiento cae de forma abrupta. Dentro del grupo correcto, el mejor desempate sigue siendo la holgura $ts - tr$, por lo que esa combinación queda empíricamente justificada como la versión más robusta del criterio voraz implementado.

4.3. Complejidad y Corrección

Complejidad. El algoritmo voraz ejecuta dos fases:

1. ordena los n tableros según el criterio $\frac{tr}{p}$ con desempate por holgura;

2. evalúa la permutación resultante.

El siguiente pseudocódigo resume esa estructura:

```
Funcion roV(tablones)
    orden <- [0,1,...,n-1]
    ordenar(orden, comparar)
    retornar evaluar_orden(tablones, orden)
Funcion comparar(i, j)
    si  $tr_i p_j \neq tr_j p_i$  retornar  $tr_i p_j < tr_j p_i$ 
    si  $ts_i - tr_i \neq ts_j - tr_j$  retornar  $ts_i - tr_i < ts_j - tr_j$ 
    retornar  $i < j$ 
```

Proposición. En el pseudocódigo anterior, el paso `ordenar(orden, comparar)` cuesta $\Theta(n \log n)$ en la implementación del proyecto.

Demostración. Sea n el número de tablones. La inicialización

$$\text{orden} \leftarrow [0, 1, \dots, n-1]$$

cuesta $O(n)$. El paso dominante del pseudocódigo es la operación de ordenamiento. En la notación del pseudocódigo, esa operación corresponde a `ordenar`. Como el comparador realiza un número constante de operaciones aritméticas y comparaciones enteras, cada comparación cuesta $O(1)$. Por tanto, para la cota superior se obtiene¹

$$T_{\text{ordenar}}(n) = O(n \log n).$$

Para la cota inferior, basta observar que el paso abstracto `ordenar(orden, comparar)` pertenece a la familia de ordenamientos por comparaciones, porque decide el orden relativo usando exclusivamente el comparador `comparar`. Ordenar n elementos distintos en ese modelo exige distinguir entre las $n!$ permutaciones posibles. Cada comparación solo tiene dos resultados, así que después de h comparaciones solo pueden diferenciarse a lo sumo 2^h casos. Por lo tanto, necesariamente debe cumplirse

$$2^h \geq n!,$$

y al tomar logaritmos se obtiene

$$h \geq \log_2(n!).$$

Ahora bien, en el producto $n! = 1 \cdot 2 \cdot \dots \cdot n$, los últimos $n/2$ factores son al menos $n/2$. Entonces

$$n! \geq \left(\frac{n}{2}\right)^{n/2},$$

¹En la implementación concreta del proyecto, el paso `ordenar(orden, comparar)` se resuelve con `std::sort`. Véase el borrador del estándar C++, sección [\[sort\]](#), donde se especifica la complejidad de `std::sort` como $O(N \log N)$ comparaciones y proyecciones.

y por tanto

$$h \geq \log_2(n!) \geq \frac{n}{2} \log_2\left(\frac{n}{2}\right) = \Omega(n \log n).$$

En consecuencia, cualquier ordenamiento por comparaciones requiere $\Omega(n \log n)$ comparaciones en el peor caso. Como ya se tenía la cota superior $O(n \log n)$, se concluye que

$$T_{\text{ordenar}}(n) = \Theta(n \log n).$$

Como la evaluación posterior de la permutación recorre los n tableros una sola vez, su costo es $O(n)$ y no modifica el orden asintótico total de **roV**. Por tanto, la complejidad temporal total del pseudocódigo, y de la implementación que lo materializa, es

$$\Theta(n \log n).$$

El espacio auxiliar es $O(n)$, debido al arreglo de índices que almacena la permutación resultante.

Correctitud del algoritmo voraz. El algoritmo voraz no siempre da la respuesta correcta al problema, entendiendo por respuesta correcta una programación de riego con costo óptimo. Aunque el problema sí tiene subestructura óptima, el criterio local usado por el algoritmo no garantiza la propiedad de escogencia voraz. Por esta razón, escoger en cada paso el tablón que parece más conveniente según $\frac{tr}{p}$ no asegura necesariamente obtener la mejor solución global.

En consecuencia, el algoritmo debe entenderse como una heurística eficiente: puede coincidir con el óptimo en ciertos tipos de instancias, especialmente cuando el retraso domina el costo, pero no es exacto para el caso general. A continuación se justifica formalmente esta afirmación mediante la subestructura óptima del problema y un contraejemplo a la escogencia voraz.

Subestructura óptima. Sí se cumple. Sea

$$\Pi^* = \langle \pi_0^*, \pi_1^*, \dots, \pi_{n-1}^* \rangle$$

una programación óptima. Si fijamos el primer tablón π_0^* , entonces el resto de la programación siempre comienza después de $tr_{\pi_0^*}$. Por tanto, el sufijo

$$\langle \pi_1^*, \dots, \pi_{n-1}^* \rangle$$

debe ser óptimo para ese subproblema residual. Si no lo fuera, existiría otro orden sobre esos mismos tableros con menor costo; al reemplazar solo el sufijo en Π^* , se obtendría una programación total más barata, contradicción.

Por ejemplo, si una solución óptima fuera $\langle 2, 0, 1, 3 \rangle$, entonces el sufijo $\langle 0, 1, 3 \rangle$ debe ser el mejor posible una vez regado T_2 . Si existiera un sufijo mejor, como $\langle 1, 0, 3 \rangle$, entonces $\langle 2, 1, 0, 3 \rangle$ costaría menos que la supuesta solución óptima. Por lo tanto, el problema sí tiene subestructura óptima.

Propiedad de escogencia voraz. No se cumple en general. Considérese la finca con dos tableros

$$T_0 = \langle 10, 2, 1, 0 \rangle, \quad T_1 = \langle 5, 3, 4, 0 \rangle.$$

Como

$$\frac{tr_0}{p_0} = 2,00, \quad \frac{tr_1}{p_1} = 0,75,$$

el criterio voraz elige primero a T_1 . Sin embargo:

Permutación	Cálculo	Costo total
$\langle 1, 0 \rangle$	$T_1 : 5 - (0 + 3) = 2, T_0 : 2(10 - (3 + 2)) = 10$	12
$\langle 0, 1 \rangle$	$T_0 : 10 - (0 + 2) = 8, T_1 : 2(5 - (2 + 3)) = 0$	8

Tabla 12: Contraejemplo a la propiedad de escogencia voraz.

La solución óptima comienza por T_0 , no por el tablero de menor $\frac{tr}{p}$. Esto demuestra formalmente que no existe garantía de que una solución óptima empiece con la escogencia voraz.

4.3.1. Síntesis e interpretación de la evidencia

Análisis de la demostración formal y de los resultados de simulación. Teoría y evidencia apuntan a la misma idea:

- el problema general no cumple la propiedad de escogencia voraz, porque la función de costo mezcla tres regímenes;
- en los casos 1 y 2 a veces conviene esperar para aprovechar rp o conservar margen;
- en el caso 3 retrasarse es caro y además el daño se multiplica por la prioridad;
- por eso no existe una regla local que sea siempre exacta;
- aun así, el argumento de intercambio de la Sección 4.1 sí justifica $\frac{tr}{p}$ cuando la instancia está dominada por tardanzas.

Síntesis de lo encontrado en los experimentos. Los experimentos refuerzan esa lectura:

- **Batería de pruebas + contraejemplo formal:** el voraz sí puede fallar; en 18 casos solo alcanzó una vez el costo óptimo y el exceso relativo promedio fue 16.23 %.
- **Bloque mixto de 5000 fincas:** la tasa de costo óptimo fue baja ($219/5000 = 4,38 \%$), pero el exceso relativo promedio bajó a 1.97 %.
- **Perfiles puros:** fue perfecto en crítico, fuerte en urgente y perdió precisión al alcanzar el costo óptimo cuando aumentó la holgura, sobre todo en relajado.
- **Comparación de 20 combinaciones:** las cuatro mejores variantes mantienen $\frac{tr}{p}$ como criterio principal; la mejor que lo abandona ya sube a 26.14 % de exceso promedio.

Cuándo acierta y cuándo falla según la estructura de costos. Con base en esa evidencia conjunta, el criterio tiende a acertar cuando la instancia está dominada por el costo de llegar tarde y falla cuando la decisión depende más de administrar margen y de coordinar el día perfecto rp :

■ **Acierta mejor cuando casi todo el riesgo está en entrar al caso 3.**

Eso ocurre en perfiles críticos y urgentes, donde el margen $ts - tr$ es nulo o pequeño. Si esperar casi siempre empeora el costo, la regla $\frac{tr}{p}$ se alinea bien con la presión real del problema.

- *Ejemplo:* si un tablón tiene $margen = 0$, cualquier espera adicional ya lo hace llegar tarde; si otro además tiene tr pequeño y prioridad alta, regarlo primero reduce rápido una penalización potencial grande.

■ **Puede elegir un orden no óptimo sin alejarse mucho del costo óptimo.**

En mezclas realistas, el voraz puede ordenar algunos tablonos de forma distinta a un orden de costo óptimo y aun así terminar cerca del mejor costo posible. Eso explica por qué en el bloque mixto la tasa de costo óptimo fue baja, pero el exceso relativo promedio siguió siendo solo 1.97 %.

- *Ejemplo:* si dos tablonos están ambos cerca del caso 3 y sus razones $\frac{tr}{p}$ son muy similares, cambiar su orden puede impedir alcanzar exactamente el costo óptimo sin producir una diferencia grande en el costo total.

■ **Falla más cuando el problema consiste en decidir quién puede esperar sin daño.**

Eso aparece en perfiles balanceados, largo plazo y sobre todo relajados, donde el margen es amplio. Allí ya no basta con evitar tardanzas; hay que decidir si conviene usar el tiempo actual o reservarlo para alinear mejor otro tablón con su día perfecto rp .

- *Ejemplo:* dos tablonos pueden seguir en caso 2 aunque esperen, pero uno tiene $rp = 2$ y el otro $rp = 8$; el mejor orden puede depender de aprovechar ese rp temprano, no de la razón $\frac{tr}{p}$.

■ **Falla especialmente cuando una decisión local desplaza a un tablón más sensible.**

Si un tablón posponible se atiende primero porque tiene buena razón $\frac{tr}{p}$, puede consumir tiempo y empujar a otro tablón hacia un peor régimen de costo. El problema no es solo evaluar mal un tablón aislado, sino no anticipar su efecto sobre los siguientes.

- *Ejemplo:* un tablón relajado puede esperar varios días sin problema, pero si se riega antes que uno urgente con $margen = 1$, ese segundo tablón puede pasar de caso 2 a caso 3 y encarecer mucho la solución.

Resumen. En relación con el problema original, la evidencia unificada permite afirmar lo siguiente:

- $\frac{tr}{p}$ no resuelve exactamente el problema general, porque la corrección depende de alcanzar un costo óptimo y ese resultado no se determina solo por urgencia y prioridad, sino también por cuándo conviene esperar.
- Aun así, $\frac{tr}{p}$ es la mejor regla local evaluada en el proyecto: fue la base de las cuatro mejores combinaciones del laboratorio comparativo y domina claramente a priorizar supervivencia, prioridad alta, margen o día perfecto como criterio principal.
- Por tanto, el voraz debe entenderse como una heurística rápida y bien fundada para aproximar el costo óptimo, especialmente útil cuando se necesita respuesta inmediata; si se requiere garantizar una solución de costo óptimo para el caso general, hace falta un método global como programación dinámica.

5. Solución Mediante Programación Dinámica

5.1. Descripción de la Estrategia

La implementación del proyecto resuelve esta parte mediante programación dinámica *top-down* con memoización sobre subconjuntos de tableros. La idea central es modelar cada subproblema por el conjunto de tableros que ya fueron regados y calcular, para ese estado, el costo mínimo restante. Esta formulación sigue los cuatro pasos clásicos de la programación dinámica.

5.1.1. Paso 1: Caracterizar la estructura de la solución óptima

En lugar de pensar directamente en permutaciones completas, la estrategia observa el problema por **estados**. Un estado queda determinado por el conjunto de tableros que ya fueron regados. Si llamamos M a ese conjunto, entonces el tiempo acumulado hasta ese punto queda completamente determinado por los tableros de M . Lo representamos por

$$\tau(M) = \sum_{i \in M} tr_i.$$

Aquí es importante distinguir dos cantidades diferentes: tr_i sigue representando la **duración del riego** del tablero i , mientras que $\tau(M)$ representa el **instante acumulado** en el que comenzaría a regarse el siguiente tablero después de completar los de M .

Con esta definición, el subproblema asociado a M puede expresarse así:

determinar el costo mínimo adicional necesario para completar el riego de todos los tableros que aún no pertenecen a M , comenzando en el instante $\tau(M)$.

La programación dinámica es aplicable porque se cumplen las dos propiedades fundamentales:

- **Subestructura óptima:** si desde un estado M la mejor decisión es regar primero el tablón i , entonces el resto del plan debe ser también óptimo para el nuevo estado $M \cup \{i\}$.
- **Solapamiento de subproblemas:** distintos órdenes parciales pueden llevar al mismo conjunto M , de modo que el mismo subproblema reaparece muchas veces si no se guarda su resultado.

5.1.2. Paso 2: Definir recursivamente el valor de la solución óptima

Para cada tablón i , definimos su costo local al comenzar a regarse en el instante t :

$$c_i(t) = \begin{cases} ts_i - (t + tr_i), & \text{si } t = rp_i, \\ 2(ts_i - (t + tr_i)), & \text{si } t \leq ts_i - tr_i \text{ y } t \neq rp_i, \\ 2p_i((t + tr_i) - ts_i), & \text{en otro caso.} \end{cases}$$

En esta expresión, t denota el **instante de inicio del riego** del tablón i , mientras que tr_i denota el **tiempo que dura** su riego. Es decir, t indica cuándo empieza el tablón y $t + tr_i$ indica cuándo termina.

Ahora definimos $DP(M)$ como el costo mínimo adicional a partir del estado M . Si ya fueron regados todos los tablonos, no queda ningún costo por agregar. En caso contrario, se prueba cada tablón que aún no ha sido regado y se elige la mejor continuación. Eso produce la recurrencia:

$$DP(M) = 0 \quad \text{si } M \text{ contiene todos los tablonos,}$$

$$DP(M) = \min_{i \notin M} \{c_i(\tau(M)) + DP(M \cup \{i\})\} \quad \text{en otro caso.}$$

En palabras: desde el estado M , el algoritmo intenta regar como siguiente tablón a cada $i \notin M$; para cada opción suma el costo inmediato de regar i en el instante actual y el mejor costo posible del subproblema que queda después.

5.1.3. Paso 3: Calcular el valor de la solución óptima

La implementación concreta usa memoización y esto se observa directamente en la función `Resolver(M)` del pseudocódigo. Su funcionamiento puede leerse en cuatro acciones:

- primero verifica si M ya contiene todos los tablonos; si eso ocurre, retorna 0, que es el caso base;
- luego consulta si `dp[M]` ya está definido; si lo está, reutiliza ese valor y evita recomputar el subproblema;

- si el estado todavía no ha sido resuelto, calcula el instante actual $t \leftarrow \tau(M)$ y recorre todos los tableros que aún no pertenecen a M ;
- para cada candidato i , evalúa el costo

$$c_i(t) + \text{Resolver}(M \cup \{i\}),$$

conserva el menor valor encontrado y finalmente lo almacena en $\text{dp}[M]$.

Así, el pseudocódigo implementa exactamente la recurrencia del paso anterior y garantiza que cada estado se calcula una sola vez.

5.1.4. Paso 4: Construir una solución óptima

La reconstrucción también puede leerse directamente en el pseudocódigo. Cada vez que $\text{Resolver}(M)$ encuentra un mejor candidato, no solo actualiza $\text{dp}[M]$, sino que guarda en $\text{eleccion}[M]$ cuál fue el tablero que produjo ese mejor costo. Después, la función roPD usa esa información de la siguiente manera:

- comienza en el estado vacío, es decir, cuando todavía no se ha regado ningún tablero;
- consulta $\text{eleccion}[M]$ para saber qué tablero debe regarse a continuación en una solución óptima;
- agrega ese tablero al vector orden y actualiza el estado a $M \cup \{i\}$;
- repite el proceso hasta que el estado contiene todos los tableros.

De esta manera, el pseudocódigo no solo calcula el costo óptimo, sino que además reconstruye explícitamente una permutación que alcanza ese costo.

El siguiente pseudocódigo resume los pasos 3 y 4 de la estrategia:

```

Funcion Resolver(M)
  si M = todos los tableros
    retornar 0
  si dp[M] esta definido
    retornar dp[M]

  t <- tau(M)
  mejor_costo <- +infinito
  mejor_siguiete <- -1

  para cada i que aun no este en M
    candidato <- c_i(t) + Resolver(M U {i})
    si candidato < mejor_costo
      mejor_costo <- candidato
      mejor_siguiete <- i

  dp[M] <- mejor_costo
  eleccion[M] <- mejor_siguiete
  retornar mejor_costo

Funcion roPD(tableros)
  inicializar dp y eleccion
  costo_optimo <- Resolver(vacio)
  M <- vacio
  orden <- []

  mientras M no contenga todos los tableros
    i <- eleccion[M]
    agregar i a orden
    M <- M U {i}

  retornar evaluar_orden_completo(tableros, orden)

```

5.1.5. Correspondencia entre estrategia e implementación

La estrategia descrita arriba se refleja directamente en la función `roPD` implementada en `[backend/src/algoritmos.cpp]`. El siguiente fragmento muestra su núcleo:

```

vector<long long> dp(limite_estados, -1);
vector<int> elecciones(limite_estados, -1);

auto resolver_dp = [&](auto& self, int mascara, int dia_actual) -> long long {
  if (mascara == limite_estados - 1) {
    return 0;
  }
  if (dp[mascara] != -1) {
    return dp[mascara];
  }

```

```

long long mejor_costo = -1;
int mejor_siguiete_tablon = -1;

for (int i = 0; i < n; ++i) {
    if ((mascara & (1 << i)) == 0) {
        long long costo_de_este = calcular_costo(tablones[i], dia_actual);
        long long costo_del_resto = self(
            self,
            mascara | (1 << i),
            dia_actual + tablones[i].tiempo_riego
        );
        long long costo_total_rama = costo_de_este + costo_del_resto;

        if (mejor_costo == -1 || costo_total_rama < mejor_costo) {
            mejor_costo = costo_total_rama;
            mejor_siguiete_tablon = i;
        }
    }
}

elecciones[mascara] = mejor_siguiete_tablon;
return dp[mascara] = mejor_costo;
};

```

Cada parte del fragmento corresponde a un elemento de la estrategia:

- `limite_estados = 1 << n` materializa que la DP trabaja sobre los 2^n subconjuntos posibles de tablones.
- `dp[mascara]` almacena el costo óptimo del subproblema asociado al estado M , es decir, al conjunto de tablones ya regados.
- `elecciones[mascara]` guarda qué tablón produce el mejor valor de la recurrencia y permite reconstruir una permutación óptima al finalizar.
- La lambda `resolver_dp(..., mascara, dia_actual)` implementa el subproblema $DP(M)$ usando tanto la máscara del estado como el instante $\tau(M)$ en el que comenzaría el siguiente riego.
- La condición `mascara == limite_estados - 1` es el caso base: si todos los tablones ya fueron regados, el costo adicional es 0.
- La prueba `dp[mascara] != -1` implementa la memoización: si el estado ya fue resuelto, el algoritmo reutiliza ese valor y evita recomputarlo.
- La condición `(mascara & (1 << i)) == 0` selecciona los tablones que todavía no pertenecen al conjunto M .
- `calcular_costo(tablones[i], dia_actual)` implementa exactamente la función local $c_i(t)$ en el instante actual.

- La llamada recursiva enciende el bit del tablón i en la máscara actual y suma su tiempo de riego al instante acumulado; eso corresponde exactamente a la transición $M \rightarrow M \cup \{i\}$.
- La actualización de `mejor_costo` y `mejor_siguiete_tablon` realiza el mínimo de la recurrencia y conserva la decisión necesaria para reconstruir la solución.

La fase de reconstrucción aparece inmediatamente después en la misma función:

```
vector<int> orden_optimo;
int mascara_actual = 0;

while (mascara_actual != limite_estados - 1) {
    int siguiente = elecciones[mascara_actual];
    orden_optimo.push_back(siguiente);
    mascara_actual |= (1 << siguiente);
}
```

Este ciclo implementa de manera literal el paso 4 de la estrategia: empieza en el estado vacío, consulta la mejor elección almacenada para ese estado, agrega el tablón correspondiente al orden y avanza al nuevo subconjunto hasta alcanzar el estado completo. Por eso la implementación no solo calcula el valor óptimo, sino que además construye explícitamente una permutación que alcanza ese costo.

5.2. Análisis de Complejidad en Tiempo y en Espacio

Proposición. La complejidad en tiempo de la programación dinámica descrita arriba es $O(n2^n)$ y su complejidad en espacio es $O(2^n)$.

Demostración. La clave del análisis está en contar cuántos estados distintos puede tomar la función `Resolver` y cuánto cuesta procesar cada uno.

Cada estado está determinado por un subconjunto $M \subseteq U$. Como U tiene n elementos, el número total de subconjuntos posibles es:

$$|\mathcal{P}(U)| = 2^n.$$

Debido a la memoización, cada estado M se resuelve a lo sumo una vez. Cuando vuelve a aparecer, el pseudocódigo retorna inmediatamente el valor almacenado en `dp[M]`. Por tanto, el costo total queda determinado por el procesamiento de esos 2^n estados distintos.

En cada estado, el bloque dominante es el ciclo

$$\text{para cada } i \in U \setminus M.$$

En el peor caso, este ciclo examina n tablonos. Cada iteración realiza solo operaciones de tiempo constante: verificar pertenencia mediante la máscara, evaluar $c_i(t)$, consultar o invocar recursivamente un estado memoizado y comparar costos. Luego, el costo por estado es $O(n)$.

En consecuencia, el costo total de calcular todos los valores de la tabla dinámica es:

$$2^n \cdot O(n) = O(n2^n).$$

La fase de reconstrucción recorre exactamente una vez los n tablonos del orden óptimo, de modo que cuesta $O(n)$. Esta fase no altera la cota dominante. Por tanto, la complejidad en tiempo total del algoritmo es:

$$O(n2^n).$$

En espacio, las estructuras dominantes son los arreglos **dp** y **eleccion**, ambos indexados por máscara y de tamaño 2^n . Además, la recursión y el vector del orden reconstruido aportan a lo sumo $O(n)$ posiciones adicionales. Desde el punto de vista asintótico, las constantes de esas tablas no alteran la cota, aunque en la estimación práctica sí importa distinguir que en la implementación real **dp** guarda costos y **elecciones** guarda índices. Por ello,

$$O(2^n) + O(n) = O(2^n),$$

y la complejidad en espacio total es:

$$O(2^n).$$

Utilidad práctica. Bajo la hipótesis del enunciado de 3×10^8 operaciones por minuto y 4 bytes por celda, y contando las dos tablas dominantes del algoritmo (**dp** y **eleccion**), la memoria base abstracta es aproximadamente $8 \cdot 2^n$ bytes. En la implementación real del proyecto, **dp** se almacena como **long long** y **elecciones** como **int**, de modo que la memoria base efectiva se aproxima mejor por $12 \cdot 2^n$ bytes. La tabla siguiente no corresponde a una corrida experimental real con $n = 32$, sino a una **proyección analítica** obtenida a partir de las cotas $O(n2^n)$ y $O(2^n)$ bajo las hipótesis del enunciado. Con esa referencia, se obtiene el siguiente panorama para $n = 2^k$:

k	$n = 2^k$	$n2^n$ operaciones	Tiempo aprox.	Memoria abstracta $8 \cdot 2^n$	Memoria real $12 \cdot 2^n$
3	8	2,048	despreciable	2 KB	3 KB
4	16	1,048,576	0.0035 min	512 KB	768 KB
5	32	137,438,953,472	458 min	32 GB	48 GB

Tabla 13: Estimación práctica de la programación dinámica, distinguiendo la memoria abstracta del análisis y la memoria base de la implementación real.

Tiempo: valores aceptables e inaceptables en la práctica. Bajo la hipótesis del enunciado, el tiempo que se tomaría el equipo en resolver el problema puede interpretarse así:

- $k = 3$ ($n = 8$): **acceptable**. El algoritmo requiere 2,048 operaciones, lo que equivale a un tiempo despreciable frente a la capacidad de 3×10^8 operaciones por minuto.
- $k = 4$ ($n = 16$): **acceptable**. El costo sube a 1,048,576 operaciones, pero eso sigue representando apenas 0,0035 minutos, es decir, una fracción muy pequeña de un minuto.

- $k = 5$ ($n = 32$): **inacceptable**. El costo alcanza 137,438,953,472 operaciones, que se traducen en aproximadamente 458 minutos. Un tiempo de varias horas deja de ser razonable para uso práctico ordinario.

Por tanto, para los valores analizados, el tiempo es **acceptable** para $k = 3$ y $k = 4$, pero se vuelve **inacceptable** a partir de $k = 5$.

Espacio: valores aceptables e inaceptables en la práctica. Bajo la misma hipótesis, el espacio que tomaría el equipo en resolver el problema puede interpretarse así:

- $k = 3$ ($n = 8$): **acceptable**. Tanto la memoria abstracta (2 KB) como la memoria base de la implementación real (3 KB) son completamente manejables.
- $k = 4$ ($n = 16$): **acceptable**. La memoria abstracta es de 512 KB y la memoria base real es de 768 KB, valores todavía pequeños en la práctica.
- $k = 5$ ($n = 32$): **inacceptable**. Incluso en la estimación abstracta ya se requieren 32 GB para las dos tablas principales, y en la implementación real esa base sube a 48 GB. Ese consumo deja de ser razonable para un entorno de trabajo ordinario.

Por tanto, para los valores analizados, el espacio es **acceptable** para $k = 3$ y $k = 4$, pero se vuelve **inacceptable** a partir de $k = 5$.

Conclusión sobre utilidad práctica. La programación dinámica es perfectamente viable para tamaños pequeños y medianos, pero deja de ser práctica cuando n supera los 20 tableros aproximadamente. A partir de $n = 32$, según esta proyección analítica, tanto el tiempo como el espacio crecen demasiado rápido. Aun así, constituye una mejora enorme frente a la fuerza bruta, cuyo crecimiento factorial la vuelve inviable mucho antes: mientras la fuerza bruta no escala más allá de $n \approx 10$, la PD alcanza cómodamente $n = 20$ y en la práctica del proyecto resolvió todas las instancias de la batería con exactitud garantizada.

5.3. Corrección

Teorema. Para toda finca F con conjunto de tableros U , la función roPD devuelve una permutación $\hat{\Pi}$ tal que

$$CR_F^{\hat{\Pi}} = \min_{\Pi} CR_F^{\Pi}.$$

Demostración. La prueba se hace por inducción sobre el número de tableros que faltan por regar.

Para cada subconjunto $M \subseteq U$, interpretamos $DP(M)$ como el costo mínimo adicional desde el estado en el que ya fueron regados exactamente los tableros de M y el instante actual es $\tau(M)$.

Caso base. Si $M = U$, no queda ningún tablero por regar. El costo adicional correcto es 0. El pseudocódigo retorna precisamente 0, de modo que $DP(U)$ es correcto.

Paso inductivo. Supongamos que para todo subconjunto M' con a lo sumo r tableros pendientes, la función **Resolver**(M') devuelve correctamente el costo óptimo adicional desde ese estado. Consideremos ahora un subconjunto M con exactamente $r + 1$ tableros pendientes.

Cualquier programación factible que continúe desde M debe escoger primero un tablero $i \notin M$. Si esa decisión se toma en el instante $\tau(M)$, el costo total de esa rama se descompone necesariamente como:

$$c_i(\tau(M)) + (\text{costo óptimo del resto desde } M \cup \{i\}).$$

Por hipótesis inductiva, el costo óptimo del resto es exactamente $DP(M \cup \{i\})$. Por tanto, para cada $i \notin M$, el mejor costo posible de una solución que empieza con i es:

$$c_i(\tau(M)) + DP(M \cup \{i\}).$$

Como toda solución factible desde M debe empezar con algún $i \notin M$, el mejor costo posible desde M es precisamente el mínimo de esas cantidades:

$$DP(M) = \min_{i \notin M} \{c_i(\tau(M)) + DP(M \cup \{i\})\}.$$

Eso es exactamente lo que calcula el pseudocódigo. Luego **Resolver**(M) devuelve el costo óptimo para todo subconjunto M con $r + 1$ tableros pendientes.

Por inducción, la recurrencia es correcta para todo subconjunto $M \subseteq U$, y en particular para $M = \emptyset$. Así, **Resolver**(vacío) produce el costo óptimo global.

Falta justificar la reconstrucción. En cada estado M , la variable **eleccion**[M] almacena un tablero que alcanza el mínimo de la recurrencia. Al seguir iterativamente esas elecciones desde el estado vacío hasta U , se construye una permutación $\hat{\Pi}$ cuyas decisiones locales coinciden con una cadena de subproblemas óptimos. Por consiguiente, el costo de $\hat{\Pi}$ es exactamente $DP(\emptyset)$, que ya se demostró igual al mínimo global.

En conclusión, **roPD** siempre devuelve una solución óptima.

5.4. Aplicación del algoritmo y evaluación experimental

5.4.1. Aplicación a los ejemplos del enunciado

Ejemplo 1. Para la finca

$$F_1 = \langle \langle 10, 3, 4, 0 \rangle, \langle 6, 3, 3, 1 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 10, 4, 2, 5 \rangle \rangle$$

la programación dinámica produce la permutación óptima:

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle.$$

Paso	Tablón	t_{inicio}	Caso aplicado	Costo
1	T_2	0	caso 1: $2 - (0 + 2)$	0
2	T_1	2	caso 2: $2(6 - (2 + 3))$	2
3	T_0	5	caso 2: $2(10 - (5 + 3))$	4
4	T_3	8	caso 3: $2 \cdot 1 \cdot ((8 + 1) - 8)$	2
5	T_4	9	caso 3: $2 \cdot 2 \cdot ((9 + 4) - 10)$	12

Tabla 14: Evaluación detallada de la PD sobre F_1 .

$$CR_{F_1}^{\Pi^*} = 20.$$

Frente al costo de 34 obtenido por el voraz, la PD reduce el costo en 14 unidades al evitar que T_2 , cuyo margen es cero, quede postergado.

Ejemplo 2. Para la finca

$$F_2 = \langle \langle 9, 3, 4, 0 \rangle, \langle 5, 3, 3, 2 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 6, 4, 2, 1 \rangle \rangle$$

la programación dinámica produce igualmente

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle \quad \text{con costo} \quad 32.$$

Frente a los 55 del voraz, el patrón de mejora es el mismo: atender primero T_2 , que tiene margen nulo, evita que entre al caso 3.

5.4.2. Evaluación experimental

En los 18 casos de la batería oficial del repositorio que cuentan con salida de referencia, la programación dinámica coincidió exactamente con el costo óptimo en todos los casos. Los valores detallados por test se presentan más adelante en las tablas comparativas de la Sección 4; por esa razón, en particular en las Tablas 15, 16 y 17, la columna **Costo PD** se usa como referencia óptima para cuantificar las desviaciones del algoritmo voraz.

Para instancias más grandes, los tiempos de ejecución crecieron de acuerdo con la complejidad exponencial del algoritmo. En particular, los casos mixtos con $n = 20$ siguieron siendo resolubles en tiempos prácticos, mientras que a partir de $n = 32$ la estrategia deja de ser viable por el crecimiento combinado del tiempo y la memoria.

6. Comparación de las tres soluciones implementadas

Para evaluar el desempeño de los tres enfoques (fuerza bruta, voraz y programación dinámica) se diseñó una batería de casos de prueba que cubre distintos tamaños de finca y diferentes perfiles de tableros. A continuación se describe el diseño de estos casos y se presentan los resultados más relevantes.

6.1. Diseño de casos de prueba

La comparación se apoyó en tres diseños de prueba con objetivos distintos:

- **Diseño 1: Casos reales del repositorio (tests 1 a 18).**
 - **Cómo se diseñaron:** se tomaron las 18 fincas oficiales del repositorio junto con sus salidas óptimas de referencia. Los tamaños cubiertos son $n = 5, 6, 7, 8, 10, 15, 20$.
 - **Propósito:** usar una batería homogénea y con respuesta conocida para comparar exactitud, cercanía al óptimo y viabilidad práctica de las tres soluciones.
- **Diseño 2: Casos adicionales diseñados manualmente (F1 a F6).**
 - **Cómo se diseñaron:** se fijó $n = 5$ para que fuerza bruta, voraz y programación dinámica pudieran ejecutarse sin restricciones. Se variaron deliberadamente urgencia, prioridad y holgura, retomando la misma lógica usada en la evaluación del voraz: $\text{margen} = ts - tr$, $rp \in [0, \text{margen}]$ y perfiles críticos, urgentes, balanceados, relajados y de largo plazo.
 - **Propósito:** observar cómo cambian los resultados cuando no solo varía el tamaño, sino también la estructura interna de la finca.
 - **Referencia:** véase la Subsección 4.2.2, donde se desarrolla el análisis experimental detallado y se reportan los resultados obtenidos con este diseño de perfiles.
- **Diseño 3: Prueba complementaria con tres criterios voraces sobre los casos reales.**
 - **Cómo se diseñó:** sobre los mismos 18 tests del repositorio se reaplicó el componente voraz con tres configuraciones: el criterio implementado $\frac{tr}{p}$ con desempate por $ts - tr$, y dos criterios alternativos tomados de la comparación de criterios de la sección del voraz: ts con desempate por $\frac{tr}{p}$ y prioridad alta con desempate por $\frac{tr}{p}$.
 - **Propósito:** distinguir si el comportamiento observado depende solo del enfoque voraz como heurística o también del criterio concreto con el que se ordenan los tablonos.
 - **Justificación:** los dos criterios alternativos se eligieron como los más relevantes dentro de la comparación de criterios de la sección del voraz para contrastarlos frente al criterio escogido sobre la batería oficial. En particular, ts con desempate por $\frac{tr}{p}$ fue la mejor opción que no utiliza $\frac{tr}{p}$ como criterio principal, mientras que prioridad alta con desempate por $\frac{tr}{p}$ aporta un contraste adicional centrado en la intuición de prioridad. Esto permite ofrecer una comparación más completa del panorama voraz sobre los casos proporcionados.

En síntesis, el Diseño 1 sostiene la comparación principal entre las tres soluciones, el Diseño 2 preserva la lógica experimental usada para entender el comportamiento del voraz y el Diseño 3 amplía la comparación interna entre criterios heurísticos.

6.2. Resultados comparativos

Para cada caso se ejecutaron los tres algoritmos:

- **Fuerza bruta (FB):** en esta comparación se reporta solo para $n \leq 8$, donde su tiempo fue razonable. Devuelve el costo óptimo.
- **Voraz (V):** siempre ejecutable; se reporta su costo.
- **Programación dinámica (PD):** ejecutable hasta $n = 20$; devuelve el costo óptimo.

Para hacer más clara la lectura, los resultados voraces se presentan en tres bloques:

- **Criterio 1:** $\frac{tr}{p}$ con desempate por $ts - tr$.
- **Criterio 2:** ts con desempate por $\frac{tr}{p}$.
- **Criterio 3:** prioridad alta con desempate por $\frac{tr}{p}$.

En esta subsección se usan dos formas de resumir el % exceso y conviene distinguirlas explícitamente:

- **Promedio global sobre los 18 tests:** se calcula promediando el % exceso de todos los casos de la batería oficial y se usa para comparar la calidad general de un criterio frente a los otros.
- **Promedio por tamaño n :** se calcula promediando solo los tests que comparten el mismo tamaño de finca; este es el valor que se grafica en las figuras de tendencia.

Criterio 1: $\frac{tr}{p}$ con desempate por $ts - tr$.

- **Tabla:** detalle por test.
- **Figura:** promedio del % exceso agrupado por tamaño n .

Test	n	Costo FB	Costo voraz	Costo PD	Diferencia (V-Ópt)	% exceso	$\hat{V} = \hat{\text{Ópt}}?$
1	5	44	59	44	15	34.09	No
2	5	126	128	126	2	1.59	No
3	5	38	57	38	19	50.00	No
4	5	26	52	26	26	100.00	No
5	6	193	193	193	0	0.00	Sí
6	6	256	288	256	32	12.50	No
7	7	67	80	67	13	19.40	No
8	7	294	302	294	8	2.72	No
9	8	143	178	143	35	24.48	No
10	8	522	534	522	12	2.30	No
11	10	–	762	698	64	9.17	No
12	10	–	638	526	112	21.29	No
13	15	–	1297	1276	21	1.65	No
14	15	–	2468	2460	8	0.33	No
15	15	–	1080	1030	50	4.85	No
16	20	–	3816	3758	58	1.54	No
17	20	–	2894	2861	33	1.15	No
18	20	–	1590	1512	78	5.16	No

Tabla 15: Comparación de costos entre los tres algoritmos usando el criterio voraz implementado $\frac{tr}{p}$ con desempate por $ts - tr$. FB se reporta solo para $n \leq 8$ (– indica no reportado en esta comparación). El costo PD se toma como referencia óptima.

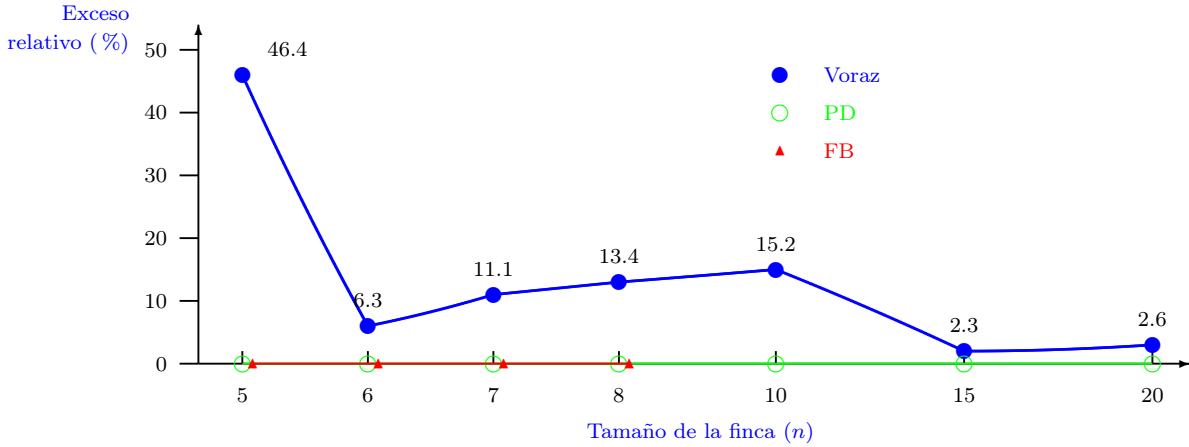


Figura 1: Exceso relativo promedio por tamaño de finca para el criterio voraz implementado $\frac{tr}{p}$ con desempate por $ts - tr$. Cada punto de la curva voraz representa el promedio del % exceso de los tests con el mismo tamaño n , redondeado a una cifra decimal.

Lectura rápida del criterio 1.

- **Exactitud de los métodos exactos.** En la Tabla 15, fuerza bruta y programación dinámica coinciden siempre con el costo óptimo en los casos donde se reportan. Esa misma idea aparece resumida en la Figura 1, donde ambas estrategias permanecen en la línea base.

- **Comportamiento del criterio implementado.** La Tabla 15 muestra que la variante $\frac{tr}{p}$ con desempate por $ts - tr$ solo alcanza el óptimo en 1 de los 18 tests y que sus mayores fallas se concentran en casos como el test 4, donde el exceso relativo llega al 100 %, y el test 12, donde la diferencia absoluta llega a 112.
- **Tendencia por tamaño.** La Figura 1 evidencia que el promedio por tamaño más alto del criterio implementado ocurre en $n = 5$, debido a varios casos pequeños muy adversos, mientras que para $n = 15$ y $n = 20$ el promedio por tamaño baja a alrededor de 2,45 %. Esto sugiere que, aunque el voraz no garantiza optimalidad, en instancias grandes puede comportarse como una heurística competitiva cuando la prioridad es responder rápido.

Criterio 2: ts con desempate por $\frac{tr}{p}$.

- **Tabla:** detalle por test para la variante alternativa.
- **Figura:** promedio del % exceso agrupado por tamaño n .

Test	n	Costo FB	Costo voraz alt.	Costo PD	Diferencia (Alt-Ópt)	% exceso	¿Alt. = Ópt?
1	5	44	76	44	32	72.73	No
2	5	126	158	126	32	25.40	No
3	5	38	60	38	22	57.89	No
4	5	26	36	26	10	38.46	No
5	6	193	277	193	84	43.52	No
6	6	256	282	256	26	10.16	No
7	7	67	200	67	133	198.51	No
8	7	294	378	294	84	28.57	No
9	8	143	205	143	62	43.36	No
10	8	522	714	522	192	36.78	No
11	10	–	1454	698	756	108.31	No
12	10	–	796	526	270	51.33	No
13	15	–	1782	1276	506	39.66	No
14	15	–	3008	2460	548	22.28	No
15	15	–	1522	1030	492	47.77	No
16	20	–	5686	3758	1928	51.30	No
17	20	–	3185	2861	324	11.32	No
18	20	–	2256	1512	744	49.21	No

Tabla 16: Comparación de costos entre los tres algoritmos usando el criterio voraz alternativo ts con desempate por $\frac{tr}{p}$. FB se reporta solo para $n \leq 8$ (– indica no reportado en esta comparación). El costo PD se toma como referencia óptima.

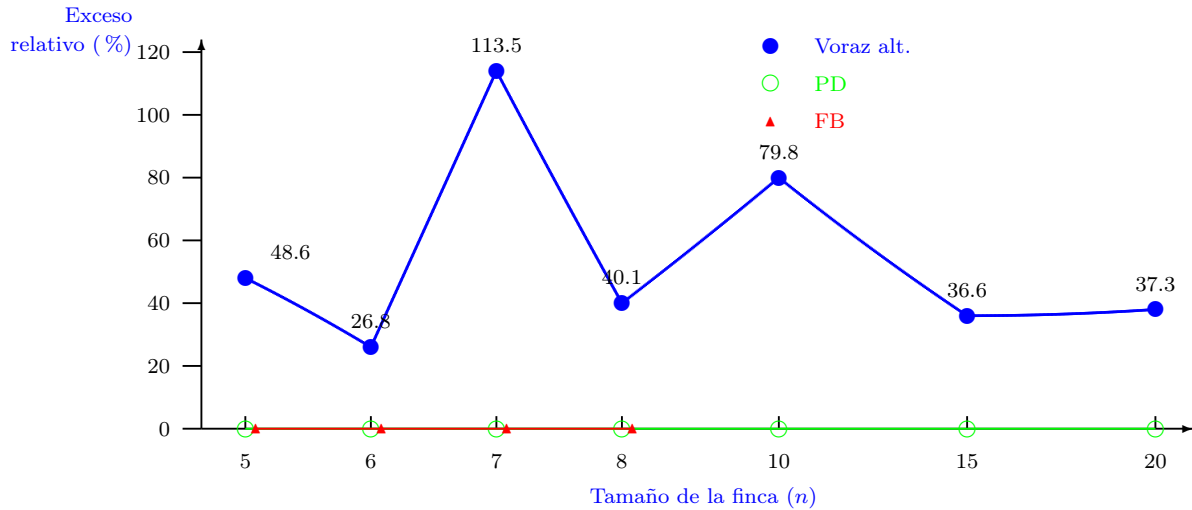


Figura 2: Exceso relativo promedio por tamaño de finca para el criterio voraz alternativo ts con desempate por $\frac{tr}{p}$. Cada punto de la curva representa el promedio del % exceso de los tests con el mismo tamaño n , redondeado a una cifra decimal.

Lectura rápida del criterio 2.

- **Sin coincidencias exactas.** La Tabla 16 muestra que esta variante no alcanza el óptimo en ninguno de los 18 tests.
- **Errores más altos.** Los casos más críticos aparecen en $n = 7$ y $n = 10$, donde la diferencia absoluta y el % exceso crecen con fuerza.
- **Tendencia por tamaño.** La Figura 2 confirma que el deterioro no es puntual: el promedio por tamaño permanece alto en casi todos los tamaños de finca.

Criterio 3: prioridad alta con desempate por $\frac{tr}{p}$.

- **Tabla:** detalle por test para la variante centrada en prioridad.
- **Figura:** promedio del % exceso agrupado por tamaño n .

Test	n	Costo FB	Costo voraz prio.	Costo PD	Diferencia (Prio-Ópt)	% exceso	¿Prio. = Ópt?
1	5	44	57	44	13	29.55	No
2	5	126	160	126	34	26.98	No
3	5	38	57	38	19	50.00	No
4	5	26	52	26	26	100.00	No
5	6	193	193	193	0	0.00	Sí
6	6	256	344	256	88	34.38	No
7	7	67	88	67	21	31.34	No
8	7	294	366	294	72	24.49	No
9	8	143	172	143	29	20.28	No
10	8	522	538	522	16	3.07	No
11	10	–	792	698	94	13.47	No
12	10	–	725	526	199	37.83	No
13	15	–	1426	1276	150	11.76	No
14	15	–	2833	2460	373	15.16	No
15	15	–	1146	1030	116	11.26	No
16	20	–	4150	3758	392	10.43	No
17	20	–	3425	2861	564	19.71	No
18	20	–	1862	1512	350	23.15	No

Tabla 17: Comparación de costos entre los tres algoritmos usando el criterio voraz alternativo prioridad alta con desempate por $\frac{tr}{p}$. FB se reporta solo para $n \leq 8$ (– indica no reportado en esta comparación). El costo PD se toma como referencia óptima.

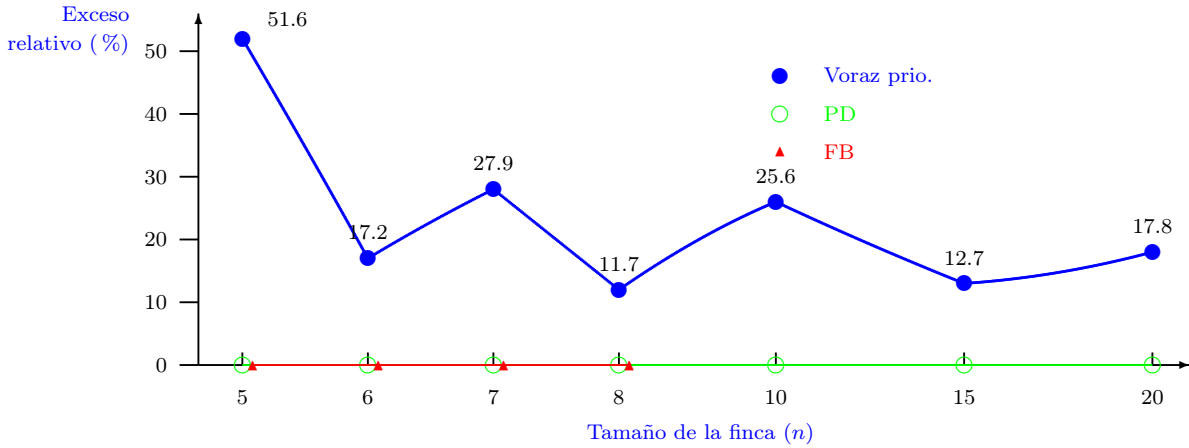


Figura 3: Exceso relativo promedio por tamaño de finca para el criterio voraz alternativo prioridad alta con desempate por $\frac{tr}{p}$. Cada punto de la curva representa el promedio del % exceso de los tests con el mismo tamaño n , redondeado a una cifra decimal.

Lectura rápida del criterio 3.

- **Una coincidencia exacta.** La Tabla 17 muestra que esta variante coincide con el óptimo en 1 de los 18 tests.

- **Resultado intermedio.** Su exceso relativo promedio global sobre los 18 tests es mayor que el del criterio implementado, pero claramente menor que el del criterio ts .
- **Tendencia por tamaño.** La Figura 3 muestra un comportamiento más estable que el criterio ts , aunque todavía sensiblemente peor que $\frac{tr}{p}$ con desempate por $ts - tr$.

Contraste entre los tres criterios voraces.

- **Mejor criterio en la batería oficial.** Entre los tres comparados, $\frac{tr}{p}$ con desempate por $ts - tr$ sigue siendo el mejor: alcanza el menor exceso relativo promedio global sobre los 18 tests (16,23 %).
- **Segundo lugar.** Prioridad alta con desempate por $\frac{tr}{p}$ ocupa una posición intermedia, con exceso relativo promedio global de 25,71 %.
- **Criterio más débil en esta batería.** El criterio ts con desempate por $\frac{tr}{p}$ resulta el más costoso en el conjunto de 18 tests, con exceso relativo promedio global de 52,03 % y ninguna coincidencia exacta con el óptimo.
- **Lectura global.** La comparación conjunta de las Tablas 15, 16 y 17, junto con las Figuras 1, 2 y 3, refuerza que el rendimiento del componente voraz depende fuertemente del criterio principal usado para ordenar los tablonos.

Aporte interpretativo del Diseño 2 a la justificación del criterio voraz.

- **Función distinta.** A diferencia de los Diseños 1 y 3, el Diseño 2 no se usa aquí para comparar criterios sobre la batería oficial, sino para interpretar el comportamiento del criterio voraz elegido.
- **Aporte a la justificación.** Mediante perfiles controlados de finca, permite identificar de forma resumida dónde la regla $\frac{tr}{p}$ con desempate por $ts - tr$ destaca, dónde se modera y dónde flaquea.
- **Lectura complementaria.** Por eso, el Diseño 2 debe leerse como apoyo breve para la elección y justificación del criterio, mientras que los Diseños 1 y 3 sostienen la comparación cuantitativa presentada en tablas y figuras.

6.3. Ventajas y desventajas de cada enfoque en la práctica

A partir de los resultados obtenidos y del análisis de complejidad, se pueden resumir las principales ventajas y desventajas de cada estrategia:

Estrategia	Complejidad en tiempo	Complejidad en espacio	Ventajas importantes	Desventajas
Fuerza bruta	$O(n! \cdot n)$	$O(n)$	Garantiza la solución óptima, es conceptualmente simple y sirve como referencia para validar instancias pequeñas.	Se vuelve inviable muy rápido y no ofrece escalabilidad práctica.
Voraz	$O(n \log n)$	$O(n)$	Es la alternativa más rápida, funciona para cualquier tamaño de entrada y en muchos casos produce costos cercanos al óptimo.	No garantiza optimalidad y su rendimiento depende de la estructura de la instancia.
Programación dinámica	$O(n2^n)$	$O(2^n)$	Garantiza la solución óptima y mejora drásticamente a la fuerza bruta en instancias moderadas.	El crecimiento exponencial en tiempo y memoria la limita a tamaños moderados.

Tabla 18: Comparación entre complejidad en tiempo, complejidad en espacio, ventajas importantes y desventajas de las tres estrategias.

7. Conclusiones

Este proyecto muestra que no existe una estrategia universalmente superior para todas las instancias del problema. La conveniencia de cada solución depende del tamaño de la finca, de la criticidad de la decisión, de la disponibilidad de recursos computacionales y del grado de error que sea aceptable en la aplicación. Desde esa perspectiva, fuerza bruta, voraz y programación dinámica no compiten solo como algoritmos, sino como respuestas distintas a necesidades también distintas.

La fuerza bruta aporta una ventaja técnica fundamental: garantiza el óptimo global y permite verificar con total certeza el comportamiento de las demás estrategias. Esa propiedad la convierte en un excelente mecanismo de validación teórica y experimental para instancias pequeñas. Sin embargo, su complejidad en tiempo $O(n! \cdot n)$ impone un límite práctico muy estricto; en este trabajo solo resultó razonable en tamaños reducidos, de modo que su papel natural queda acotado a servir como referencia exacta, no como solución operativa escalable.

La programación dinámica representa la alternativa adecuada cuando la información es crítica y obtener una solución óptima justifica un mayor consumo de tiempo y memoria. La formalización del problema mediante subestructura óptima y recurrencia permite construir una solución exacta con fundamentos teóricos sólidos, y en el diseño de pruebas mostró un desempeño correcto en los tamaños moderados considerados. No obstante, su complejidad en tiempo $O(n2^n)$ y su complejidad en espacio $O(2^n)$ obligan a delimitar cuidadosamente el tamaño del problema: en este proyecto se mantuvo como opción viable hasta alrededor de $n = 20$, pero a mayor escala también termina volviéndose computacionalmente inviable.

El enfoque voraz, por su parte, ofrece la mejor respuesta cuando la eficiencia es prioritaria y se necesita procesar instancias grandes con rapidez. Su costo $O(n \log n)$ lo hace claramente más liviano que la búsqueda exhaustiva y que la programación dinámica, aunque esa ventaja exige aceptar que no siempre producirá el óptimo. Los experimentos mostraron precisamente que la naturaleza del problema juega un papel decisivo: según el perfil de los tablonos, el criterio elegido puede destacar, moderarse o fallar. Aun así, tanto el análisis teórico como la evidencia experimental justifican que una sobreestimación controlada del costo puede ser aceptable cuando se compara con el precio computacional de aplicar un método exacto.

En conjunto, los resultados permiten sostener que la decisión algorítmica debe tomarse con criterios explícitos: hasta qué tamaño puede crecer la instancia, cuánto importa la optimalidad exacta, cuáles recursos computacionales están disponibles y qué nivel de desviación puede tolerarse. La programación dinámica muestra el valor de formalizar rigurosamente la estructura del problema para acercarse a la mejor solución posible, mientras que el voraz demuestra que, aun cuando la naturaleza de la instancia no siempre favorece la voracidad, sigue siendo una opción técnicamente valiosa cuando se necesitan respuestas rápidas y bien fundamentadas. Ese equilibrio entre teoría, modelación y validación experimental es, en última instancia, la principal enseñanza que deja el proyecto.

Notas de implementación

Esta nota resume de forma breve cómo está organizada la implementación del proyecto y cómo se conectan sus componentes principales.

- **Núcleo algorítmico.** La implementación de las tres estrategias solicitadas se encuentra en `backend/src/algoritmos.cpp`. Allí se definen las funciones `roFB`, `roV` y `roPD`, cada una encargada de recibir una finca y devolver una solución compuesta por la permutación de riego y su costo total.
- **Tipos de dominio.** Las estructuras básicas del problema, como `Tablon`, `ResultadoCalculo` y los datos auxiliares de salida, se declaran en `backend/include/domain_types.hpp`. Esto permite que el cálculo algorítmico y la capa de interfaz compartan un mismo modelo de datos.
- **Lectura de entradas.** La transformación del texto de entrada hacia una finca interna se realiza en `backend/src/placeholder_engine.cpp`. La primera línea del archivo indica n , y cada una de las siguientes n líneas usa el formato `ts, tr, p, rp`.
- **Capa de servicio.** El backend expone rutas HTTP desde `backend/src/main.cpp`. En particular, la ruta `/calcular` recibe el algoritmo solicitado y el texto de la finca, ejecuta `roFB`, `roV` o `roPD`, y responde en formato JSON con costo, permutación y detalles por tablón.
- **Interfaz.** La interfaz principal está en `riego_interfaz_frontend.html` y en los módulos de `assets/js`. Desde allí se puede cargar una finca, visualizar sus tablonos, escoger el algoritmo, revisar el resultado calculado y exportar la salida.

- **Formato de salida.** Para efectos de exportación, la salida mantiene la estructura pedida en el enunciado: primera línea con el costo total y luego n líneas con los índices de los tableros en el orden en que deben ser regados.
- **Pruebas y validación.** Los casos de prueba del proyecto se encuentran en `bateria_pruebas/`. Las entradas y salidas oficiales del repositorio se usaron para validar la exactitud de fuerza bruta y programación dinámica, y para medir la desviación del algoritmo voraz frente al costo óptimo.